



Project: Book Rating Prediction, Bookly

Instructor: Hanna Abi Akl

Project Report

Group Members

Full Name	Tasks
TAMEEM ANSARI	Feature Engineering and Model Training (Data Science)
KADAPPAGARI VENUMADHAV	Model Training, Testing and Evaluation (Data Science)
FOTSO TEMEU Samuel Daryl	EDA, Data Cleaning, Feature Engineering, and Model Training (Data Engineering)
NALLAPULI YASHASWINI	EDA and Web Application (Data Analysis)

DSTI School of Engineering, August 2026

Introduction

This report covers our work on the Book Rating Prediction project for the Machine Learning with Python course. The assignment gave us a raw dataset of 11,128 books, each with 12 attributes: an identifier, title, authors, average rating, ISBN in two formats, language code, page count, number of ratings, number of text reviews, publication date and publisher. The task was to predict a book's average rating from its metadata and to package the result as a small web application, following the same pipeline structure we used in class on the Titanic dataset: explore, clean, engineer features, train and compare models, then deploy.

Predicting a rating from catalogue information only is a harder problem than it looks. A book's rating is mostly decided by things that are not in this dataset at all, the writing itself, whether the story lands with people, word of mouth, marketing, timing. What we do have is a proxy signal through the publisher and the author: certain publishers and certain authors consistently produce books that rate well, and that turned out to be the strongest thing our model could learn from. We treated this as a regression problem, since `average_rating` is a continuous value roughly between 1 and 5, not a category, and we picked our models and metrics accordingly.

The sections below follow the order we actually worked in: first we looked at the data to understand what we were dealing with, then cleaned it, then built the features the model would use, then compared several models and picked the best one, and finally wrapped it in a Streamlit application. All the numbers quoted in this report come directly from our notebooks, we did not round anything up to make the results look better.

Methodology

Exploratory Data Analysis

Before writing any cleaning or modelling code we loaded the file as-is and looked at what we had. A handful of rows in the CSV have unquoted commas inside the title (for example a subtitle containing a comma), which breaks the default parser, so we loaded the file with pandas' python engine and skipped the malformed lines, which brought us from 11,128 to 11,119 usable rows. We also had to strip the column names, since `num_pages` arrives from the source file with leading spaces.

We started with the target itself. `average_rating` is not spread evenly across 0 to 5, most books sit between 3.0 and 4.5, and the distribution is left skewed, which makes sense: a book that is genuinely bad rarely accumulates enough ratings to even appear in a dataset like this one. This told

us early on that the model would likely be better at predicting an average book than an outstanding or a terrible one, which is exactly what we saw later in the residuals.

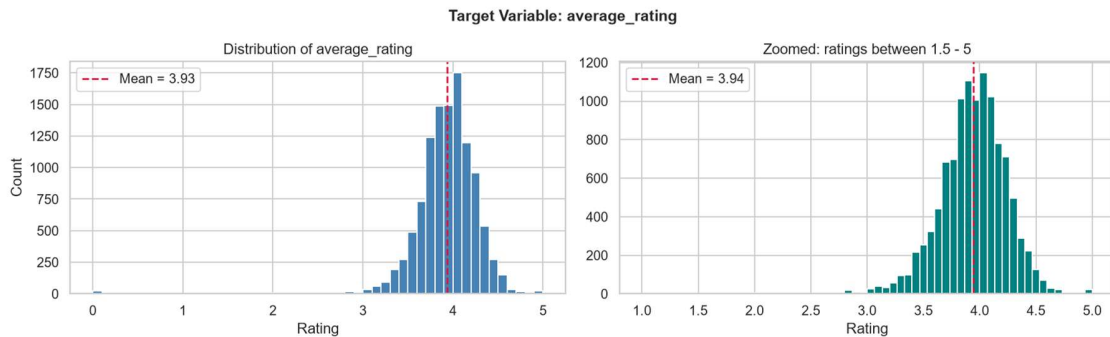


Figure 1. Distribution of `average_rating` across the raw dataset.

We checked how each raw numeric column relates to the rating. `bookID` and `isbn13` came back with a correlation close to zero (-0.035 and -0.002), which is expected since they are just identifiers assigned by whoever compiled the dataset, and we dropped both for that reason. `num_pages` had the best correlation of any single raw feature at around 0.15 . `ratings_count` and `text_reviews_count` looked almost useless in their raw form, but that turned out to be a scale problem rather than a signal problem, since these two columns run from 0 to several million, with a tiny number of extremely popular books completely dominating the range. Once we applied a log transform during feature engineering the correlation roughly doubled, which we cover in the next section.

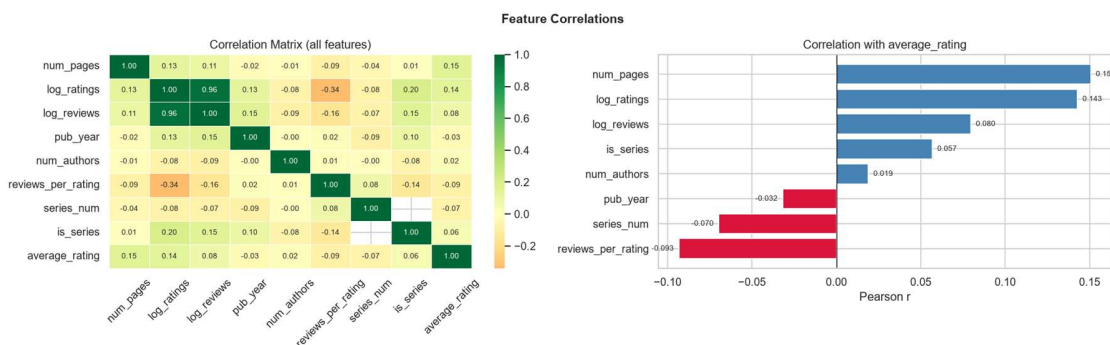


Figure 2. Correlation matrix and correlation of each numeric feature with the target, computed after preliminary log transforms.

We also grouped the data by language and by publisher to check for patterns. English dominates the dataset heavily, and once grouped by language the average rating differences were small. Publisher was a different story, grouping by publisher and looking at the mean rating showed real, sizeable differences between publishers, which is the first hint that publisher reputation would end up being a useful signal, something we confirmed formally later with target encoding.

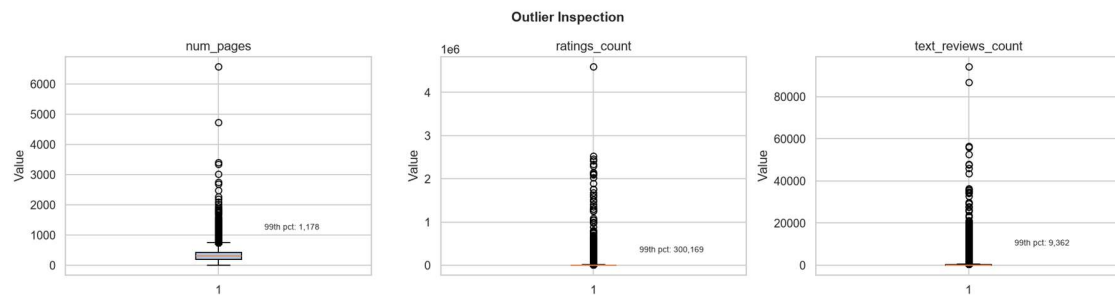


Figure 3. Boxplots of `num_pages`, `ratings_count` and `text_reviews_count` showing the extent of the skew.

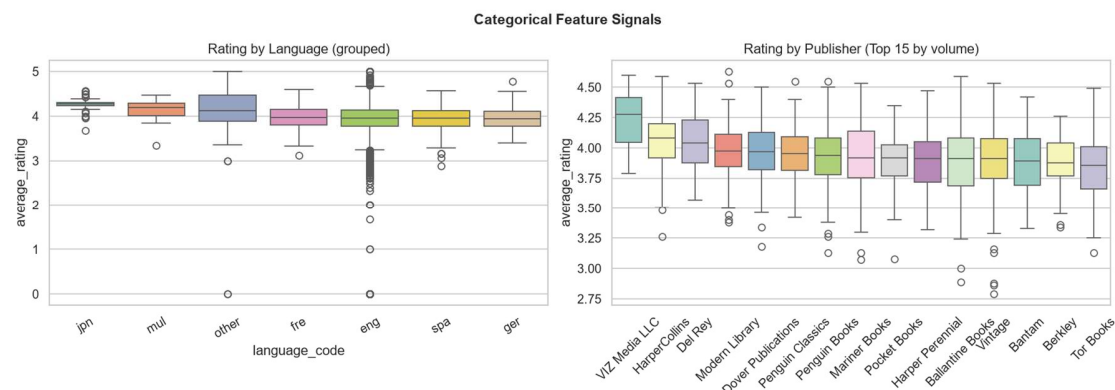


Figure 4. Rating spread across the main languages and across the twelve most frequent publishers.

Data Cleaning

Once we understood the shape of the problem, we moved to cleaning. We dropped `bookID`, `isbn` and `isbn13`, since none of them carry predictive signal and keeping them would just let a model memorise rows instead of learning anything generalisable.

About 80 books had `ratings_count` equal to 0. Some of those also had `average_rating` equal to 0, meaning there is simply no rating data for that book, and a smaller number had a non-zero rating with zero ratings behind it, which cannot be a real average. In both cases the target value cannot be trusted, so we removed those 80 rows rather than trying to guess a rating for them.

75 books had `num_pages` equal to 0, which is not a real value, a book cannot have zero pages, this is clearly a missing value that was stored as 0 instead of blank. We imputed those with the median page count across the rest of the dataset, 302 pages, rather than the mean, because page count is right skewed and a few very long books would have pulled the mean upward.

`publication_date` came in as a string such as 9/16/2006 and we parsed it down to just the year, since the exact day rarely matters for a rating prediction and the year gives us a reasonable proxy for how the book has aged. Two rows had a date that does not exist on any calendar, such as day 31 in a month that only has 30 days, and those two rows were dropped since the date could not be recovered.

The `language_code` column had 26 distinct values, but the English variants (en-US, en-GB, en-CA, and enm for Middle English) are functionally the same language, so we merged them into a single eng category. We kept the next most frequent languages, spanish, french, german, japanese, chinese, and the mul code used for multi-language editions and anthologies, and grouped every remaining rare code under other, ending with 8 categories: eng (10,473 books), spa (212), fre (140), ger (96), jpn (45), other (40), mul (19), zho (14).

One value looked suspicious at first: two books were listed with publisher 10/18, which reads like a date rather than a publisher name. We checked it and it is in fact a real French publisher, Union Générale d'Éditions 10/18, so we left those rows untouched rather than treating them as an error. We also found 308 rows that share the same title and author, which at first glance look like duplicates, but they are different editions of the same book from different publishers or in different languages, and each one carries real, distinct information for the model, so we kept all of them.

After all of the steps above, the dataset went from 11,119 rows and 12 columns down to 11,039 rows and 9 columns: title, authors, average_rating, language_code, num_pages, ratings_count, text_reviews_count, publisher and the new pub_year column.

Feature Engineering and Selection

This is the step where a mistake is easy to make without noticing it, so we want to explain the order we followed and why. Two of our columns, publisher and authors, are text with thousands of distinct values, far too many for one-hot encoding. The natural fix is target encoding, replacing each publisher or author with the average rating of their books. The problem is that if you compute that average using the full dataset and only split into train and test afterwards, the test rows have already contributed to their own encoding, and the model ends up being evaluated on numbers that quietly contain the answer. This is called data leakage, and it makes a model look far better than it is. To avoid this, we split the data into train and test first, 8,831 books in train and 2,208 in test (an 80/20 split, random_state 42), and every encoding used from that point on was computed using the training rows only and then mapped onto the test rows.

From the title we pulled three numbers: whether the title contains a series marker such as (... #6), which number in the series it is, and how many words are in the title. 1,881 of the 8,831 training books turned out to be part of a numbered series. We had discussed early on whether we could use the fact that a later book in a series (say, book 6) comes after an earlier one (book 5) as useful ordering information, but a plain regression model looks at each row independently, it has no memory of other rows, so it cannot use that kind of sequential relationship directly. What it can use is the series number itself as a plain feature, which is what we did.

From authors we counted how many people wrote each book, from 1 up to 51 for one large anthology, and separately built the target encoding described above, using training data only, with one extra detail: when a book has more than one author we average the individual author encodings rather than treating the whole author string as one category, so co-authored books are handled sensibly.

ratings_count and text_reviews_count were log-transformed with log1p, which compresses the huge raw range (up to 4,597,666 ratings for the most popular book) down to a scale between roughly 0 and 15 that sits comfortably next to the rest of the features, without which these two columns would have dominated every model purely because of their size, not because they are more informative.

For the target encodings, we used a smoothing factor of 10, meaning a publisher's encoded value is a blend between that publisher's own average rating and the overall average, weighted by how many books that publisher has in the training set. A publisher with 300 books keeps close to its own average, since there is enough evidence to trust it, while a publisher with a single book gets pulled back toward the global average, since one data point is not enough evidence on its own. We ended up with 2,001 encoded publishers, ranging from VIZ Media at 4.228 down to niche study-guide publishers around 3.68, and 7,752 encoded authors, with Stephen King, who appears in 99 books in the dataset, landing at 3.9886. In the test set, 264 books had a publisher never seen in training and 480 had every listed author unseen, and those fell back to the global training mean of 3.9469, which is the correct, honest thing to do when the model genuinely has no information about a category.

language_code was one-hot encoded into 7 binary columns with eng left as the implicit baseline. We then ran a quick 50-tree Random Forest purely to rank feature importance, not as our final model, and used it to drop anything contributing almost nothing. author_enc alone accounted for about 60 percent of the importance, publisher_enc around 12 percent, followed by log_ratings_count, num_pages, log_text_reviews_count, pub_year, title_words, series_num and num_authors in smaller amounts. is_series and all 7 language columns scored under 0.005 and were removed, since keeping them would only add noise and training time without helping accuracy. We ended up with 9 final features: num_pages, pub_year, series_num, title_words, num_authors, log_ratings_count, log_text_reviews_count, publisher_enc and author_enc.

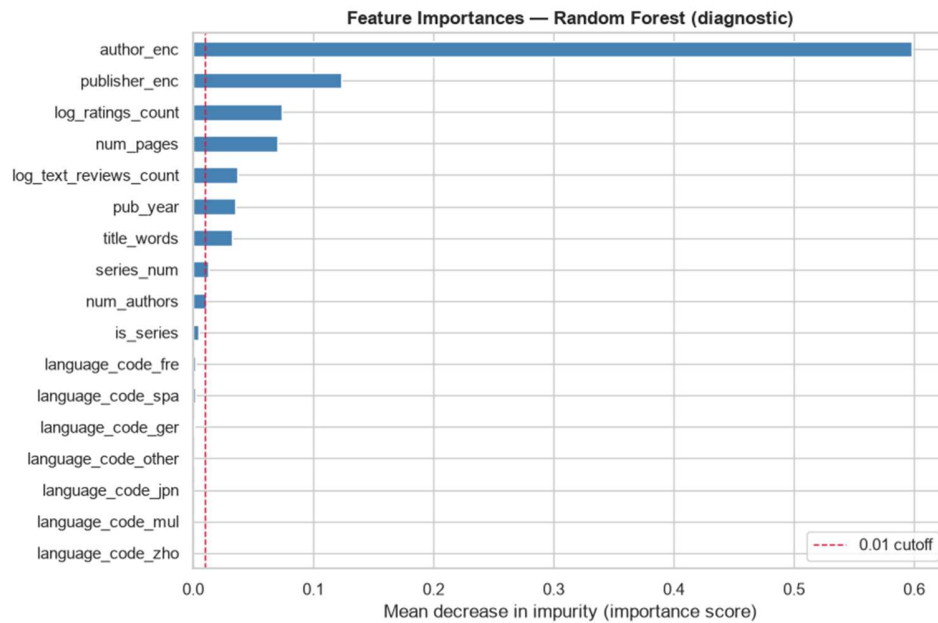


Figure 5. Feature importance from the diagnostic Random Forest, used to select the final 9 features.

Model Selection, Comparison and Evaluation

Before training anything we computed a naive baseline, always predicting the training mean regardless of input, which gave an RMSE of 0.2996 on the test set. Any model that cannot beat this number is not learning anything useful, so this is the bar every real model had to clear.

We trained six regressors: Linear Regression, Ridge, a Decision Tree, Random Forest, Gradient Boosting, and K-Nearest Neighbours with $k=7$. Every model was wrapped inside a scikit-learn Pipeline that applies StandardScaler before the model itself. This matters more than it sounds, since a Pipeline guarantees the scaler is fit only on the training fold, even when it is used inside cross-validation, so we never accidentally let test statistics leak into the scaling step the way we were careful to avoid with the target encodings.

Model	Test RMSE	Test MAE	Test R ²
Linear Regression	0.2588	0.1888	0.2509
Ridge ($\alpha = 1$)	0.2588	0.1888	0.2509
Decision Tree	0.2674	0.1905	0.2005
Random Forest	0.2576	0.1844	0.2580
Gradient Boosting	0.2576	0.1837	0.2580
KNN ($k = 7$)	0.2632	0.1891	0.2252
Naive baseline (mean)	0.2996	-	0.0000

All six models beat the baseline, which was expected, but the margins between them are fairly tight, telling us that most of the predictive power in this dataset comes from the features themselves (mainly `author_enc` and `publisher_enc`) rather than from which algorithm we picked. Random Forest and Gradient Boosting were essentially tied and clearly ahead of the rest.

To check that these numbers were not just a lucky split, we ran 5-fold cross-validation on the training set for every model.

Model	CV RMSE (mean)	CV RMSE (std)
Linear Regression	0.2043	± 0.0067
Ridge ($\alpha = 1$)	0.2043	± 0.0067
Decision Tree	0.1843	± 0.0064
Random Forest	0.1666	± 0.0057
Gradient Boosting	0.1670	± 0.0045
KNN ($k = 7$)	0.1904	± 0.0073

The low standard deviations across folds told us the results were stable and not dependent on which books happened to land in which fold. Random Forest and Gradient Boosting kept their lead here too, so we tuned only those two with `GridSearchCV` rather than spending time tuning models that were never going to win.

Model	Best parameters	CV RMSE	Test RMSE	Test MAE	Test R ²
Random Forest	<code>n_estimators=200,</code> <code>max_depth=20</code>	0.1671	0.2573	0.1840	0.2596
Gradient Boosting	<code>n_estimators=200,</code> <code>learning_rate=0.05,</code> <code>max_depth=5</code>	0.1672	0.2573	0.1835	0.2598

After tuning, Random Forest and Gradient Boosting came out essentially identical, tied on RMSE at 0.2573 and within a fraction of a point on R². We picked Random Forest as our final model, since it edged out Gradient Boosting marginally on MAE and trains noticeably faster with these settings, but we want to be upfront that the choice between the two was close enough that Gradient Boosting would be an equally defensible pick.

Results

Our final model is a tuned Random Forest Regressor with 200 trees and a maximum depth of 20, trained on the 9 features described above. On the held-out test set it reached an RMSE of 0.2573, an MAE of 0.184, and an R² of 0.2596, a 14.1 percent improvement in RMSE over the naive baseline of 0.2996.

In plain terms, our predictions are typically off by around 0.18 to 0.26 stars from the true rating. Given that the ratings themselves mostly sit in a narrow band between 3 and 4.5, being off by roughly a fifth of a star is a genuinely useful estimate, close enough to give a reader a sense of whether a book is likely to be well received without pretending to know exactly how it will land.

The R^2 of 0.26 deserves an honest explanation rather than being brushed aside. It means our model accounts for about a quarter of the variation in book ratings, the remaining three quarters comes from things this dataset does not capture at all, the actual quality of the writing, whether the story resonates with a particular reader, marketing push, timing, or plain word of mouth. We are predicting from catalogue metadata only, we have no access to review text or reader-level interaction data the way a platform like Goodreads itself would, so a moderate R^2 here is not a sign that something went wrong, it is a fair reflection of how much a rating can be guessed from title, author, publisher and a handful of counts alone.

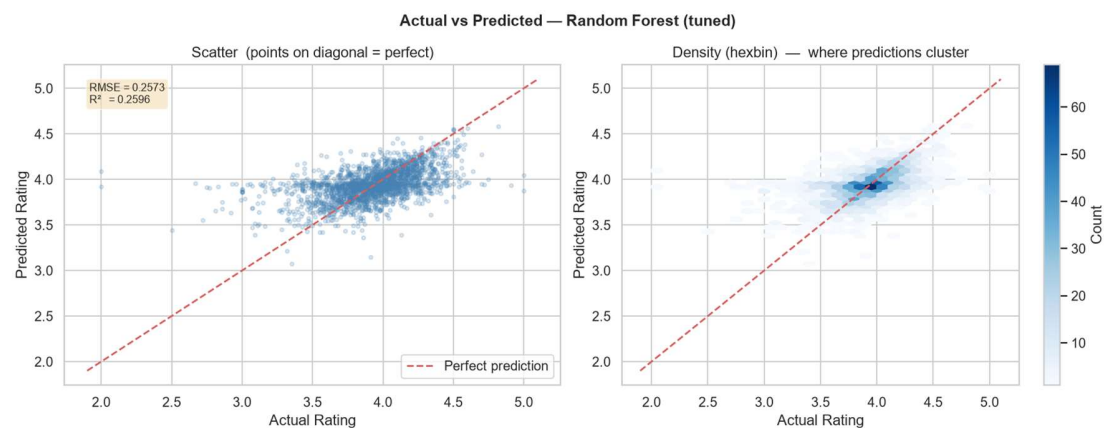


Figure 6. Predicted rating against actual rating on the test set for the final model.

The actual-versus-predicted plot shows the predictions clustering close to the diagonal in the middle of the rating range, where most books sit, and spreading out more at the edges. The residual plots confirm this: the errors are centred close to zero, meaning the model is not systematically over- or under-predicting on average, but the model tends to pull very low and very high ratings toward the middle. This is a normal property of regression models known as regression to the mean, the model plays it safe and leans toward the average whenever the features do not strongly point one way or the other, and with an R^2 this size that is exactly the behaviour we would expect.

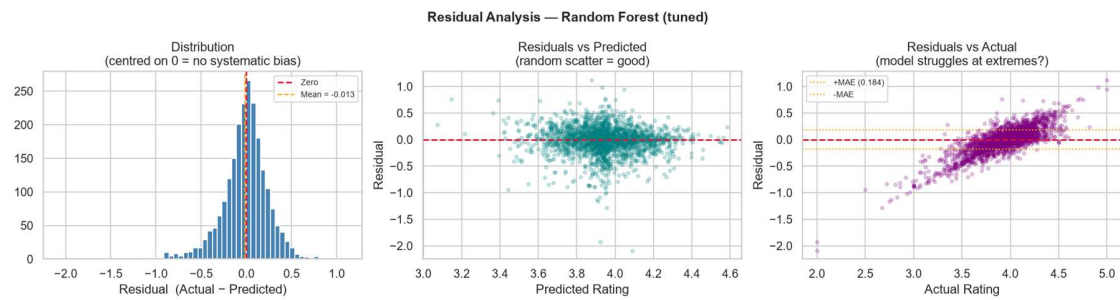


Figure 7. Residuals (actual minus predicted) plotted against predicted and actual values.

The learning curve, which trains the model on progressively larger slices of the training data, tells a slightly different story from a textbook case. The validation RMSE drops from around 0.19 to about 0.17 as more training data is added and is levelling off, while the training RMSE stays low and flat around 0.07 throughout. The gap between the two lines is a mild sign of overfitting, the tuned Random Forest (max_depth of 20 is fairly deep) fits the training rows more closely than it generalises to new ones, though the validation curve flattening out tells us that simply adding more rows of the same 9 features would only close that gap a little further. A shallower tree or a stronger regularisation setting could trade a bit of training accuracy for a tighter gap, which is a reasonable next tuning step, but the features themselves remain the bigger lever, no amount of retuning depth will hand the model information it does not have, such as genre or review text.

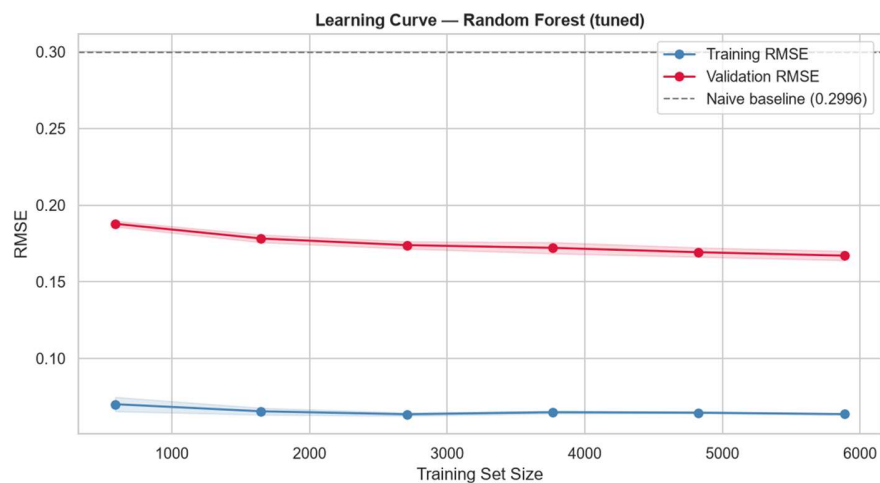


Figure 8. Learning curve for the tuned Random Forest, training and validation RMSE against training set size.

Feature importance told a clear story: author_enc alone drives most of the model's predictions, far ahead of everything else, which matches something intuitive about reading habits, readers pick up books by authors they already trust, so an author's track record is a strong proxy for how their next book will be received. publisher_enc came second, reflecting that certain publishers consistently work with stronger material or apply a more careful editorial process. Page count and how widely

a book has been read (through the log-transformed counts) mattered, but far less than author and publisher reputation.

Deployment

We built the prediction interface as a Streamlit application named Bookly. The application has two modes. The first is a single-book form where a user fills in the page count, number of ratings, number of text reviews, publication year, language, publisher and author, and gets back a predicted rating along with a plain-language confidence band. The second is a batch mode, where a CSV of several books can be uploaded at once and the app returns the same table with an added `predicted_rating` column, downloadable as a new CSV.

The prediction logic lives in a separate module, `predictor.py`, used by both modes, so a single book typed into the form and a row inside an uploaded CSV always go through the exact same transformation pipeline: the same log transforms, the same publisher and author lookups against the saved encoding maps, and the same feature ordering the model expects. Splitting this out from the interface code means the two modes cannot drift apart from each other or from how the model was trained.

The application loads five files saved directly from the training notebook: `bookly_model.pkl`, the trained and tuned pipeline itself, `publisher_encoding.pkl` and `author_encoding.pkl`, the smoothed encoding maps built from the training set, `global_mean.pkl`, the fallback value used for any publisher or author the model never saw during training, and `feature_columns.json`, the exact column order the model expects, so a typed publisher name or an uploaded CSV row is converted into numbers the exact same way every time, with no retraining needed to serve a prediction.

Deploy

Bookly

BOOK RATING PREDICTOR · DSTI SCHOOL OF ENGINEERING

Estimate the average rating a book will earn from its catalog details. The model was trained on roughly 11,000 books and reads the same signals a librarian might: who wrote it, who published it, how long it is, and how widely it has been read.

[Single book](#) [Upload a CSV](#)

Book details

Title	① Number of pages
The Night Circus	387 - +
Author(s)	② Number of ratings
Erin Morgenstern	250000 - +
Publisher	Number of text reviews
Penguin Books	15000 - +
	Publication year
	2011
	Language
	eng

☐ This book is part of a numbered series

Predict rating

3.92

/ 5.00

★★★★½☆

Typical error is around 0.18 stars, so the real rating most likely falls between 3.66 and 4.18.

Bookly predicts an average rating from catalog metadata alone. It cannot read the book, so taste, writing quality, and word of mouth are outside its view. Treat the number as an informed estimate, not a verdict.

Figure 9. Bookly, single-book prediction mode, running locally.

The complete project, including the notebooks, the app, the saved model, and this report, is available on GitHub at <https://github.com/samitochi04/bookly>. A short screen recording of the application running with a sample input and its returned prediction is included with the submission alongside this report.

Conclusion

We set out to build a complete pipeline from a raw, messy dataset to a working prediction tool, and we think Bookly does that honestly. The biggest lesson for us was less about which algorithm

to pick, since Random Forest, Gradient Boosting, and even plain Ridge all landed close to each other, and more about the features feeding those models. Nearly all of the predictive power came from two engineered signals, author and publisher reputation, built through target encoding, and getting that encoding right, splitting before touching the target and smoothing thinly-supported categories, mattered far more to the final result than which regressor we ultimately chose.

If we had more time, the clearest next step would be adding information this dataset does not contain: genre or subject tags, which we expect would meaningfully separate books that currently look similar on paper, and review text, which could be summarised with basic NLP to capture sentiment the numeric columns miss entirely. A collaborative filtering approach using reader-level interactions, the way large platforms such as Goodreads operate internally, would likely outperform anything we can build from catalogue metadata alone, but that would need a completely different kind of dataset than the one we were given.

Working across three tracks, data analysis, data science and data engineering, on the same dataset was useful in a very practical way, each of us naturally leaned into a different part of the pipeline while still needing to understand the whole thing well enough to defend it, which we think comes through in this report.

Sources of Inspiration

- Class lectures with Hanna Abi Akl, Machine Learning with Python, DSTI School of Engineering, the pipeline structure demonstrated in class on the Titanic dataset (explore, clean, engineer, train, evaluate, deploy) is the same structure we followed here.
- Joel Grus, Data Science from Scratch, used for a from first principles understanding of how the models we called from scikit-learn actually work under the hood, rather than treating them as black boxes.
- StatQuest with Josh Starmer, "Random Forests Part 1, Building, Using and Evaluating", https://www.youtube.com/watch?v=J4Wdy0Wc_xQ, used to understand how a Random Forest aggregates individual trees before we tuned its hyperparameters.